Recuperación de la Unidad I — Aplicaciones Web

Nombre: Cristhian Azael Alvarado Escamilla
Grupo: 3er C
Carrera: TSU en Desarrollo de Software Multiplataforma
Actividad: Cuestionario de recuperación

Cuestionario

1. ¿Cuál es la diferencia entre HTML, CSS y JavaScript dentro de tu portafolio? Explica una tarea específica realizada con cada tecnología.

HTML sirve para organizar el contenido de una página web, como los títulos, textos, imágenes y secciones. CSS se utiliza para darle diseño, colores, tamaños y distribución a los elementos. JavaScript permite agregar interacciones. En mi portafolio utilicé HTML para crear las secciones, CSS para diseñar las tarjetas con fondo oscuro y detalles azul turquesa, y JavaScript para que los botones de competencias muestren u oculten información.

2. ¿Qué ventajas ofrece una estructura semántica con etiquetas como `<header>`, `<nav>`, `<main>`, `<section>` y `<footer>` frente al uso indiscriminado de `<div>`?

Las etiquetas semánticas permiten identificar la función de cada parte de una página. Por ejemplo, `<header>` representa el encabezado, `<nav>` contiene la navegación, `<main>` agrupa el contenido principal, `<section>` identifica una sección y `<footer>` representa el pie de página. Esto hace que el código sea más ordenado, fácil de entender y mantener. En cambio, usar solamente `<div>` no indica claramente qué función tiene cada elemento.

3. ¿En qué se distinguen los atributos `id` y `class` de HTML? Muestra un ejemplo de selector CSS adecuado para cada uno.

El atributo `id` identifica un elemento específico de la página y debe ser único dentro del documento. El atributo `class` permite asignar una misma clase a varios elementos para aplicarles estilos similares.

Ejemplo de HTML:

```
<section id="competencias">
  <article class="tarjeta">
    <h3>HTML</h3>
  </article>
</section>
```

Ejemplo de CSS:

```
#competencias {
  padding: 30px;
}
```

```
.tarjeta {
  border-radius: 15px;
  padding: 20px;
}
```

En este ejemplo, `#competencias` selecciona el elemento por su `id`, mientras que `.tarjeta` selecciona los elementos que tienen esa clase.

4. Explica cómo se aplican Flexbox o CSS Grid para organizar tarjetas responsivas. ¿Qué ajuste harías para pantallas pequeñas?

Flexbox y CSS Grid permiten organizar elementos de manera ordenada. Flexbox facilita distribuir tarjetas en filas o columnas, mientras que Grid permite crear una distribución mediante filas y columnas. Para que las tarjetas sean responsivas, se puede utilizar CSS Grid con `auto-fit` y `minmax()`, de modo que se acomoden según el espacio disponible.

Ejemplo:

```
.contenedor {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
```

Para pantallas pequeñas haría que las tarjetas ocuparan el ancho disponible, reduciría los espacios si fuera necesario y comprobaría que no aparezca desplazamiento horizontal.

5. ¿Cómo funciona el efecto parallax en un fondo y qué consideraciones debes aplicar para garantizar legibilidad y buen comportamiento en dispositivos móviles?

El efecto parallax crea la sensación de profundidad cuando el fondo se desplaza a una velocidad diferente a la del contenido durante el desplazamiento de la página. Para aplicarlo correctamente, es importante elegir una imagen adecuada, colocar una capa oscura si hace falta y mantener suficiente contraste para leer los textos. En dispositivos móviles se debe comprobar que el fondo se vea correctamente y ofrecer una alternativa sin el efecto cuando el dispositivo no lo soporte o cuando afecte la experiencia de navegación.

6. ¿Qué es el DOM y cómo puede JavaScript seleccionar un elemento y responder a un evento click? Presenta un ejemplo sencillo.

El DOM es la representación de los elementos HTML que el navegador organiza para que JavaScript pueda consultarlos y modificarlos. JavaScript puede seleccionar un elemento mediante métodos como `getElementById()` y responder a un clic utilizando `addEventListener()`.

Ejemplo:

```
<button id="boton">Mostrar mensaje</button>
<p id="mensaje" hidden>Hola, este es mi portafolio.</p>

<script>
  const boton = document.getElementById("boton");
  const mensaje = document.getElementById("mensaje");

  boton.addEventListener("click", () => {
    mensaje.hidden = !mensaje.hidden;
  });
</script>
```

Cuando se presiona el botón, JavaScript cambia el estado de la propiedad `hidden` y muestra u oculta el mensaje.

7. ¿Qué diferencias hay entre ocultar un elemento con CSS y modificar su contenido o estado mediante JavaScript? Da un caso relacionado con el carrusel o modal.

CSS controla principalmente la apariencia de los elementos. Por ejemplo, la propiedad `display: none` permite ocultar visualmente un elemento. JavaScript permite cambiar el contenido o el estado de la página como respuesta a una acción del usuario.

En mi portafolio, los botones de las competencias utilizan JavaScript para mostrar u ocultar los detalles. De manera similar, un modal puede abrirse cuando el usuario presiona un botón y cerrarse cuando presiona la opción correspondiente. CSS se encarga de su presentación, mientras que JavaScript controla cuándo se abre o se cierra.

8. ¿Por qué son importantes las rutas relativas y una estructura ordenada de carpetas para imágenes, audios, videos, CSS y JavaScript al publicar con GitHub Pages?

Las rutas relativas permiten que una página encuentre sus archivos a partir de la ubicación del propio proyecto. Por ejemplo, `styles/styles.css` indica la ubicación de la hoja de estilos dentro de la carpeta correspondiente.

Tener las carpetas ordenadas facilita encontrar y mantener imágenes, audios, videos, archivos CSS y archivos JavaScript. También ayuda a evitar errores cuando se publica el sitio en GitHub Pages. Si una ruta está mal escrita, un archivo puede dejar de mostrarse o funcionar. Por eso es importante respetar los nombres, las extensiones y la ubicación de los archivos.

9. Explica la función de `git add`, `git commit`, `git push` y `git merge`. ¿Por qué conviene trabajar en una rama de práctica antes de integrar los cambios a `main`?

`git add` prepara los archivos modificados para incluirlos en un commit. `git commit` guarda un registro de los cambios en el historial local. `git push` envía los commits al repositorio remoto. `git merge` integra los cambios de una rama en otra.

Trabajar en una rama de práctica, como `Recuperacion_U1`, permite desarrollar y probar los cambios sin modificar directamente `main`. Cuando todo funciona correctamente, se puede integrar la rama a `main`.

Esto ayuda a organizar el trabajo, conservar el historial y reducir el riesgo de afectar la versión principal del proyecto.

10. ¿Cuál es el procedimiento general para publicar o actualizar un sitio mediante GitHub Pages y cómo comprobarías que sus enlaces, recursos multimedia e interacciones funcionan correctamente?

Primero guardaría los cambios y comprobaría que la página funciona localmente. Después prepararía los archivos con `git add`, registraría los cambios con `git commit` y enviaría los commits con `git push`. Una vez revisada la rama de trabajo, integraría los cambios a `main` y verificaría la configuración de GitHub Pages para publicar el sitio desde la rama y carpeta correctas.

Para comprobar el resultado, abriría el enlace público y revisaría que la navegación lleve a las secciones correctas, que las imágenes, audios, videos y estilos carguen, y que los botones, el carrusel y el modal respondan correctamente. También probaría el sitio en computadora y celular para detectar problemas de diseño o navegación.